

一维 Riemann 问题 MacCormack 格式：代码细节

邵桂博、杨镇宇
计算流体力学 Project 2

2026 年 7 月 5 日

摘要

这部分报告主要介绍 `1-D_Riemann_NM_MacC_02.c` 的关键代码逻辑。报告重点说明三件事：一是程序中各类变量怎样保存；二是 MacCormack 格式、滚动存储、快照输出和人工粘性在代码中怎样实现；三是数值解怎样与 Project 0 的精确解文件进行对比。

关键词：变量 struct 分组整理；滚动存储节省 CPU；快照输出"Snapshot"；调用已有 `project_0`

目录

1	当前版本与旧版本代码的区别	3
2	程序总体思路	3
2.1	一个时间步的大致流程	3
2.2	主程序做了什么	4
3	变量保存方式	4
3.1	守恒量	4
3.2	原始量	5
3.3	通量	5
3.4	不同时间层	6
3.5	求解器总结构	7
3.6	内存分配	7
4	时间推进实现	8
4.1	CFL 时间步	8
4.2	人工粘性滤波 (filter)	8
4.3	两种 MacCormack 写法	9

目录	2
4.4 关键迭代代码	9
4.5 固定使用和交替使用	11
4.6 边界条件	11
5 与 Project 0 精确解程序的连接	12
5.1 调用方式	12
5.2 为什么要传同一个网格	12
5.3 路径和引号	12
6 输出文件和后处理	12
6.1 数值解和精确解文件	13
6.2 与 Python 后处理的关系	13

1 当前版本与旧版本代码的区别

旧代码位于 `Project_2\old_code_and_data`。

本程序最开始的 old 版本主要目标是先把 MacCormack 格式跑通，因此变量命名和数组组织方式比较直接。程序中大量使用 `q1`、`q2`、`q3`、`f1`、`f2`、`f3` 这类数组名，其中 `q1`、`q2`、`q3` 分别表示守恒量中的密度、动量和总能量。这样的写法和数学推导中的分量形式比较接近，初期实现较快，但当程序加入人工粘性、不同算例、不同 CFL 数和不同差分方向之后，代码的维护和阅读变得非常痛苦，变量数量明显增加，阅读和检查代码时容易混淆不同数组的物理意义。

在 02 版本中，程序的主要改动不是更换数值格式，而是重新整理变量的组织方式。让 Code 读起来不那么痛苦（尽管变量名和方法 Method 很多）

2 程序总体思路

目前程序对应的源文件 `src` 是

`1-D_Riemann_NM_MacC_02.c`。

它求解一维无粘可压 Euler 方程：

$$\frac{\partial Q}{\partial t} + \frac{\partial F(Q)}{\partial x} = 0, \quad Q = (\rho, \rho u, \rho E)^T.$$

这份代码主要用于 Riemann 初值问题，例如 Sod shock tube。初始时刻左右两边给定不同的 (ρ, u, p) ，中间在 x_0 处有一个间断。程序把左右原始量转换成守恒量，然后用 MacCormack 格式向前推进。

2.1 一个时间步的大致流程

顺序是：

$$Q^n \rightarrow \bar{Q}^n \rightarrow \tilde{Q}^{n+1} \rightarrow Q^{n+1}.$$

这里：

- Q^n ：当前时刻的守恒量；
- $\bar{Q}^n$ ：加人工粘性滤波后的状态；
- $\tilde{Q}^{n+1}$ ：预测步得到的状态；
- Q^{n+1} ：校正步后的新状态。

程序中的主循环基本就是反复调用：

```
1 advance_one_maccormack_step(&solver);
```

每调用一次，就完成一个时间步。

2.2 主程序做了什么

main () 的执行顺序如下：

1. 建立一个带默认参数的 Solver。
2. 选择 Riemann 算例，例如 Sod、Lax 或纯接触间断。
3. 输入计算域、网格数、CFL、终止时间和人工粘性参数。
4. 给所有数组分配内存。
5. 初始化守恒量 $\rho, \rho u, \rho E$ 。
6. 进入时间推进循环。
7. 输出 Tecplot 数据，并调用 Project 0 生成精确解。

这份代码最主要的整理工作是把 CFD 变量按物理意义和时间层次放好。这样后面检查程序时，不需要反复把 q1/q2/q3 和具体物理量对应起来。同时分配释放内存也更加有规矩，很好的防止内存误删或者过度分配内存

3 变量保存方式

最开始写这类 CFD 程序时，很容易把所有数组都直接放在一个结构体里，例如 q1、q2、q3、f1、f2、f3。这种写法能运行，但读代码时经常需要回头查注释。02 版改成了按物理量分组保存。

3.1 守恒量

守恒量用 Conserved 保存：

```

1 typedef struct {
2     double *rho;
3     double *rhoul;
4     double *rhoE;
5 } Conserved;

```

它对应

$$\mathbf{Q} = (\rho, \rho u, \rho E)^T.$$

这样写之后, rho、rhoul、rhoE 的含义比较直观, 比 q1/q2/q3 更容易检查。

3.2 原始量

原始量用 Primitive 保存:

```

1 typedef struct {
2     double *rho;
3     double *u;
4     double *p;
5     double *E;
6     double *a;
7 } Primitive;

```

这些量不是单独推进的变量, 而是每一步由守恒量算出来:

$$u = \frac{\rho u}{\rho}, \quad p = (\gamma - 1) \left(\rho E - \frac{1}{2} \rho u^2 \right), \quad a = \sqrt{\gamma p / \rho}.$$

其中声速 a 用来计算 CFL 时间步, ρ, u, p 也会用于人工粘性系数的计算。

3.3 通量

通量用 Flux 保存:

```

1 typedef struct {
2     double *mass;
3     double *momentum;
4     double *energy;
5 } Flux;

```

它对应 Euler 方程中的

$$F(Q) = \begin{bmatrix} \rho u \\ \rho u^2 + p \\ u(\rho E + p) \end{bmatrix}.$$

这样写以后，质量通量、动量通量、能量通量可以直接从变量名判断。

3.4 不同时间层

MacCormack 格式中一个时间步内会出现多个中间状态。代码中用 TimeLayers 统一保存：

```

1 typedef struct {
2     Conserved current;
3     Conserved filtered;
4     Conserved predicted;
5     Conserved corrected;
6 } TimeLayers;

```

含义如下：

代码变量	数学符号	含义
current	Q^n	当前时间层
filtered	$\bar{Q}^n$	人工粘性滤波后
predicted	$\tilde{Q}^{n+1}$	预测步结果
corrected	Q^{n+1}	校正步结果

每完成一个时间步，代码把 corrected 复制回 current：

```
1 conserved_copy(solver, &solver->state.current,  
2               &solver->state.corrected);
```

这样下一步仍然从 `current` 开始，时间层关系比较明确。

3.5 求解器总结构

所有参数和数组最后放在 `Solver` 中：

```
1 typedef struct {  
2     Grid1D grid;  
3     GasModel gas;  
4     RiemannInitialCondition initial;  
5     TimeControl time;  
6     ArtificialViscosity av;  
7     MacCormackControl maccormack;  
8  
9     TimeLayers state;  
10    FluxLayers flux;  
11    Primitive primitive;  
12    double *nu;  
13 } Solver;
```

这里 `grid` 管网格，`gas` 管 γ ，`time` 管时间步，`av` 管人工粘性，`maccormack` 管差分方向。这样虽然结构体多了一些，但每一块变量的用途比较明确。

3.6 内存分配

每类数组都有对应的分配和释放函数，例如：

```

1 static int conserved_alloc(Conserved *q, size_t n);
2 static void conserved_free(Conserved *q);

```

这样做的目的是减少漏分配、漏释放数组的可能，也少写重复代码。比如一个 `Conserved` 里面总是有三条数组，分配和释放时就应该一起处理。

4 时间推进实现

本章说明一个时间步内代码具体做了什么。对应的主函数是：

```

1 advance_one_maccormack_step(&solver);

```

它里面的主要顺序如下：

```

1 solver->time.dt = compute_dt_from_cfl(solver);
2 apply_artificial_viscosity_filter(solver);
3 compute_flux_from_conserved(solver, &solver->state.filtered,
4                             &solver->flux.filtered);
5 compute_maccormack_predictor(solver, stencil);
6 compute_flux_from_conserved(solver, &solver->state.predicted,
7                             &solver->flux.predicted);
8 compute_maccormack_corrector(solver, stencil);

```

4.1 CFL 时间步

显式格式的时间步的确定方法 (以满足 CFL 条件)：

$$\Delta t = CFL \frac{\Delta x}{\max_i (|u_i| + a_i)}.$$

代码先从当前守恒量反算 ρ, u, p, E, a ，然后扫描所有网格点求 $\max_i (|u_i| + a_i)$ 。

4.2 人工粘性滤波 (filter)

根据课堂给出的滤波公式，先对当前守恒量做一次平滑：

$$\bar{Q}_i^n = Q_i^n + \frac{1}{2} \nu_i (Q_{i+1}^n - 2Q_i^n + Q_{i-1}^n).$$

其中

$$\nu_i = CFL_{\text{actual}}(1 - CFL_{\text{actual}})\beta\theta_i.$$

因为最后一个时间步可能被截短，所以这里用的是实际 CFL：

$$CFL_{\text{actual}} = \max_i (|u_i| + a_i) \frac{\Delta t}{\Delta x}.$$

老师要求保留三种人工粘性算法，所以 θ_i 可以用 ρ 、 u 或 p 来算。统一抽象写成 ϕ_i ：

$$\theta_i = \frac{|\phi_{i+1} - 2\phi_i + \phi_{i-1}|}{|\phi_{i+1} - \phi_i| + |\phi_i - \phi_{i-1}| + \epsilon}.$$

代码中通过输入选项选择 sensor：

```
1 VISCOSITY_SENSOR_RHO = 1
2 VISCOSITY_SENSOR_U = 2
3 VISCOSITY_SENSOR_P = 3
```

如果关闭人工粘性，程序直接令 $\bar{\mathbf{Q}}^n = \mathbf{Q}^n$ ，后面的 MacCormack 步骤仍然可以正常执行。

4.3 两种 MacCormack 写法

本程序支持两种常见写法。第一种是预测步用 FTBS，校正步用 FTFS：

$$\begin{aligned}\tilde{\mathbf{Q}}_i^{n+1} &= \bar{\mathbf{Q}}_i^n - \lambda [\mathbf{F}(\bar{\mathbf{Q}}_i^n) - \mathbf{F}(\bar{\mathbf{Q}}_{i-1}^n)], \\ \mathbf{Q}_i^{n+1} &= \frac{1}{2} (\bar{\mathbf{Q}}_i^n + \tilde{\mathbf{Q}}_i^{n+1}) - \frac{\lambda}{2} [\mathbf{F}(\tilde{\mathbf{Q}}_{i+1}^{n+1}) - \mathbf{F}(\tilde{\mathbf{Q}}_i^{n+1})].\end{aligned}$$

第二种是预测步用 FTFS，校正步用 FTBS：

$$\begin{aligned}\tilde{\mathbf{Q}}_i^{n+1} &= \bar{\mathbf{Q}}_i^n - \lambda [\mathbf{F}(\bar{\mathbf{Q}}_{i+1}^n) - \mathbf{F}(\bar{\mathbf{Q}}_i^n)], \\ \mathbf{Q}_i^{n+1} &= \frac{1}{2} (\bar{\mathbf{Q}}_i^n + \tilde{\mathbf{Q}}_i^{n+1}) - \frac{\lambda}{2} [\mathbf{F}(\tilde{\mathbf{Q}}_i^{n+1}) - \mathbf{F}(\tilde{\mathbf{Q}}_{i-1}^{n+1})].\end{aligned}$$

其中

$$\lambda = \frac{\Delta t}{\Delta x}.$$

4.4 关键迭代代码

从代码实现上看，一个完整时间步的核心顺序如下。前面先根据 CFL 条件确定 Δt ，然后依次完成人工粘性滤波、预测步、校正步，最后把 `corrected` 复制回 `current`，作为下一步的初值。

```

1 stencil = stencil_for_current_step(solver);
2
3 apply_artificial_viscosity_filter(solver);
4 compute_flux_from_conserved(solver, &solver->state.filtered,
5                             &solver->flux.filtered);
6 compute_maccormack_predictor(solver, stencil);
7
8 compute_flux_from_conserved(solver, &solver->state.predicted,
9                             &solver->flux.predicted);
10 compute_maccormack_corrector(solver, stencil);
11
12 conserved_copy(solver, &solver->state.current,
13               &solver->state.corrected);
14 solver->time.t += solver->time.dt;
15 solver->time.step_count++;

```

以默认的 FTBS/FTFS 写法为例，预测步中守恒量的更新代码为：

```

1 for (int i = 1; i < solver->grid.nx - 1; i++) {
2     predicted->rho[i] = filtered->rho[i]
3         - lambda * (flux->mass[i] - flux->mass[i - 1]);
4     predicted->rhov[i] = filtered->rhov[i]
5         - lambda * (flux->momentum[i] - flux->momentum[i - 1]);
6     predicted->rhoE[i] = filtered->rhoE[i]
7         - lambda * (flux->energy[i] - flux->energy[i - 1]);
8 }

```

对应的校正步使用相反方向的通量差分：

```

1  for (int i = 1; i < solver->grid.nx - 1; i++) {
2      corrected->rho[i] =
3          0.5 * (filtered->rho[i] + predicted->rho[i])
4          - 0.5 * lambda * (flux->mass[i + 1] - flux->mass[i]);
5      corrected->rhoe[i] =
6          0.5 * (filtered->rhoe[i] + predicted->rhoe[i])
7          - 0.5 * lambda * (flux->momentum[i + 1] - flux->momentum[i]);
8      corrected->rhoE[i] =
9          0.5 * (filtered->rhoE[i] + predicted->rhoE[i])
10         - 0.5 * lambda * (flux->energy[i + 1] - flux->energy[i]);
11 }

```

如果选择 FTFS/FTBS 写法，程序会进入另一个分支，预测步和校正步中的通量差分方向互换。

4.5 固定使用和交替使用

程序运行时可以选择四种模式：

1. 每一步都使用 FTBS/FTFS;
2. 每一步都使用 FTFS/FTBS;
3. 两种写法交替使用，并且第一步用 FTBS/FTFS;
4. 两种写法交替使用，并且第一步用 FTFS/FTBS。

交替模式通过当前步数的奇偶性判断本步使用哪一种写法。这样可以比较单一写法和交替写法对数值结果的影响。

4.6 边界条件

当前边界条件为简单的零梯度外推：

$$Q_0 = Q_1, \quad Q_{N_x-1} = Q_{N_x-2}.$$

滤波状态、预测状态和校正状态都会各自处理边界，避免差分时报到没有定义的边界值。

5 与 Project 0 精确解程序的连接

为了检查数值解是否合理, Project 2 在输出数值解之后, 会调用之前 Project 0 中写好的 Riemann 精确解程序。这样做的好处是不用在 Project 2 里再写一遍精确解算法, 两个程序各自负责一件事。

5.1 调用方式

Project 0 的可执行文件是

`_Analysical_Solution_Solver\riemann_exact.exe.`

Project 2 把左右状态、 γ 、计算域、网格点数、输出时间和输出文件名都拼成命令行参数, 然后通过

```
1 system(command);
```

启动精确解程序。

如果精确解程序不存在, Project 2 会给出编译提示; 如果外部程序返回错误, Project 2 也会停止当前计算。

5.2 为什么要传同一个网格

在对比精确解和数值解时, 如果直接用过去 `project_0` 跑的数据, 那么为了匹配 x 轴而插值, 也会带来误差, 这样就无法很好的对比数值解和精确解了。如果每次要手动去 `project_0` 对齐数据, 显著增加不必要的工作量。

Project 2 会把自己的

$$x_{\min}, \quad x_{\max}, \quad x_0, \quad N_x, \quad \gamma$$

传给 Project 0。这样 Project 0 生成的 `exact` 文件和 `numerical` 文件在同一组

$$x_i = x_{\min} + i \frac{x_{\max} - x_{\min}}{N_x - 1}$$

上取值。后面比较误差时就可以直接逐点比较, 不需要再做插值。

5.3 路径和引号

因为是在 Windows 下运行, 路径中可能有空格, 所以命令字符串里需要对可执行文件和输出文件名加引号。这一步主要是为了保证 `system()` 能正确找到外部程序。

6 输出文件和后处理

02 版程序输出 Tecplot ASCII 数据。输出函数是 `write_tecplot()`。文件开头的变量名为:

```
1 VARIABLES = "x", "rho", "u", "p", "E",  
2           "rho_conserved", "rhoul", "rhoE", "nu"
```

其中 ρ 、 u 、 p 、 E 是原始量， $\rho_{\text{conserved}}$ 、 ρ_{oul} 、 ρE 是守恒量， ν 是人工粘性滤波系数。

6.1 数值解和精确解文件

程序最终会输出一个 numerical 文件，并自动生成对应的 exact 文件。例如：

numerical_02.dat $\rightarrow$ numerical_02_exact.dat.

如果设置了 snapshot interval，程序也会在中间时刻输出快照，并且为每个快照调用一次精确解程序。

6.2 与 Python 后处理的关系

已有 Python 后处理脚本主要读取 x 、 ρ 、 u 、 p 。02 版虽然多输出了几列，但这几个基本变量名仍然保留，所以按变量名读取的脚本仍然可以使用。